



**DATA  
MANAGEMENT**

# DATA MANAGEMENT LAB

DATA ACQUISITION

1

Data Acquisition:  
Web Scraping

2

Data Acquisition:API

3

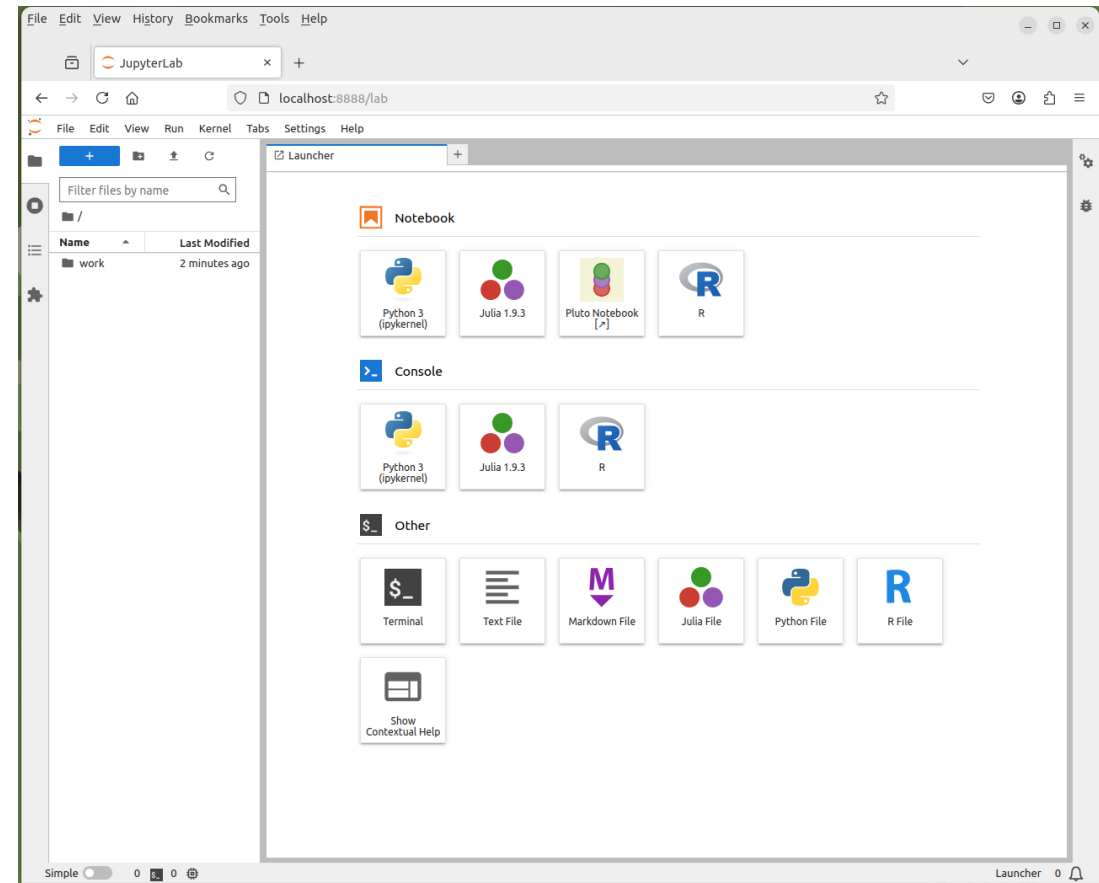
Data Acquisition:  
“real” time with  
KAFKA

4

Exercise

# HOW TO ACCESS TO YOUR VM AND USE JUPYTER

- Horizon platform
- <https://vdi-libaas.si.unimib.it/>
  - use your campus credentials
- Select the lab
- Open a terminal and type
  - `su – podman` (password podman99!)
  - `cd virtuallab`
  - `./start_lab.sh`
- Open a browser
  - <https://localhost:8888/lab?> (password lab)



# RUNNING LABS ON YOUR MACHINE

- FIRST TIME
- Install what is necessary by following the instructions provided on e-learning
- Download Docker for your operative system
  - <https://docs.docker.com/get-started/get-docker/>
- Create an empty folder called DataManLab
- Download docker compose and Dockerfile from e-learning image-lab-PC
- Unzip into the directory DataManLab
- Start docker
- Open powershell terminal, go to DataManLab
  - `docker buildx build -t virtuallab -f Dockerfile.env .`
- Docker will create virtual environments (aka container)
  - Virtuallab-dev (python, jupyter, duckdb, ..)
  - MongoDB
  - Neo4j
  - Kafka
- To use the container
- Open powershell , go to DataManLab
- Type `docker compose -f docker-compose.yaml up .`
- Open the browser and go to `https://localhost:8888/tree`



# DATA ACQUISITION



# HOW WE CAN EXTRACT DATA FROM WEB PAGES?

- Performing web scraping with:
- Text pattern matching
- HTML parsing
- DOM parsing
- ...
- We will use BeautifulSoup, a Python library to extract data from HTML/XML
- If you don't know what HTML/XML/CSS are, please see the following links:
- <https://developer.mozilla.org/en-US/docs/Web/HTML>
- [https://developer.mozilla.org/en-US/docs/Web/XML/XML\\_introduction](https://developer.mozilla.org/en-US/docs/Web/XML/XML_introduction)
- <https://developer.mozilla.org/en-US/docs/Web/CSS>

# BEAUTIFUL SOUP

- It is a Python library for pulling data out of HTML and XML files
- It works with your several parser (Python's HTML parser, lxml, html5lib) to provide idiomatic ways of navigating, searching, and modifying the parse tree
- It commonly saves programmers hours or days of work
- It provides 4 main Python objects to deal with:
  - Tag
  - NavigableString
  - Comment
  - BeautifulSoup
- Source: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>

# BEAUTIFUL SOUP: THE BEAUTIFULSOUP OBJECT

- It represents the parsed document as a whole
- For most purposes, you can treat it as a Tag object, so it supports most of the methods for searching and navigating
- It has no name and no attributes...but sometimes it is useful to look at its `.name`, so it is given the special name “document”



# BEAUTIFUL SOUP: THE TAG OBJECT

- It corresponds to an XML or HTML tag in the original document
- It has a lot of attributes and methods
- The most important features are name and attributes
- Every tag has a name, accessible as .name
- It may have any number of attributes:
- You can access a tag's attributes by treating the tag like a dictionary
- You can access that dictionary directly as .attrs:



```
html body#$(page.bodyid).$(page.bodyclass) div#container.outer.jb_home div#content_enclosure div#cont
  </dl>
  </div>
</div>
<div id="content_enclosure">
  <div id="content">
    <ul id="products_pan">
      <li class="idea">
        <h4><a href="idea/"><span>IntelliJ IDEA</span><br/>
          <span class="desc">The Most Intelligent Java IDE</span>
        </a></h4>
      </li>
      <li class="mps">
        <h4><a href="mps/"><span>MPS</span><br/>
          <span class="desc">Meta Programming System</span>
        </a></h4>
      </li>
      <li class="webstorm">
      </li>
    </ul>
```

# BEAUTIFUL SOUP: THE NAVIGABLESTRING OBJECT

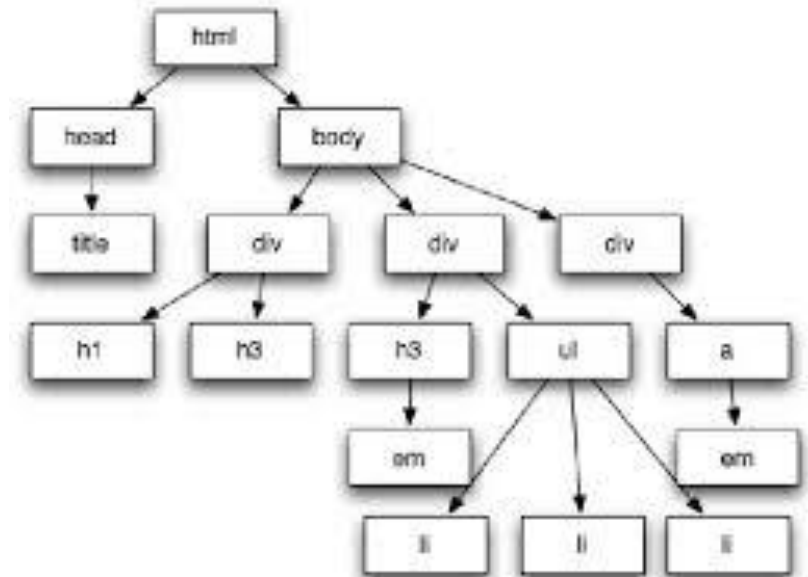
- It corresponds to the text within a tag
- It is just like a Python Unicode string, with some further features to enable tree search and navigation
- You can convert a NavigableString to a Unicode string with `unicode()` function

# BEAUTIFUL SOUP: THE COMMENT OBJECT

- It is just a special type of NavigableString
- When it appears as part of an HTML document, a Comment is displayed with special formatting ( <!-- I'm a comment --> )

# BEAUTIFUL SOUP: NAVIGATING THE TREE

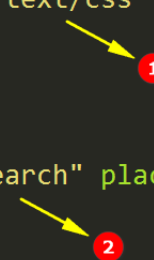
- Tags may contain strings and other tags! These elements are the tag's children.
- BeautifulSoup provides a lot of different attributes for navigating and iterating over a tag's children:
  - `.contents`
  - `.children`
  - `.descendants`
- You can also navigate up...to tag's parent:
  - `.parent`
  - `.parents`
- What about tags at the same level?!
  - `.siblings`
- See: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/#navigating-the-tree>



# BEAUTIFUL SOUP: SEARCHING THE TREE

- BeautifulSoup defines a lot of methods for searching the parse tree and they are all very similar among them.
- `find()`
- `find_all()`
- `find_parent()` and `find_parents()`
- `find_all_next()` and `find_next()`
- ...
- You can search by tag, by CSS class, by tag's attributes and many more...
- See:  
<https://www.crummy.com/software/BeautifulSoup/bs4/doc/#searching-the-tree>

```
<!DOCTYPE html>
<html>
  <head>
    <title>Wikitechy Search Form</title>
    <link rel="stylesheet" type="text/css" href="search-style.css">
  </head>
  <body>
    <form>
      <h2>HTML Search Form</h2>
      <input type="text" name="search" placeholder="Search..">
    </form>
  </body>
</html>
```



# BEAUTIFUL SOUP EXAMPLE (EXCERCISE0.IPYNB)

## ■ Perequisite

`pip install requests beautifulsoup4 lxml pandas`

## ■ The code

```
import requests
```

```
import pandas as pd
```

```
from bs4 import BeautifulSoup
```

```
URL = "https://en.wikipedia.org/wiki/2024%E2%80%9325_Serie_A"
```

```
headers = {"User-Agent": "Mozilla/5.0 (educational-webscraping-lab)"}
```

```
#Download page
```

```
response = requests.get(URL, headers=headers)
```

```
response.raise_for_status()
```

```
soup = BeautifulSoup(response.text, "lxml")
```

en.wikipedia.org/wiki/2024–25\_Serie\_A

### Stadiums and locations [ edit ]

| Team          | Location | Stadium                                        | Capacity |
|---------------|----------|------------------------------------------------|----------|
| Atalanta      | Bergamo  | Gewiss Stadium                                 | 24,950   |
| Bologna       | Bologna  | Stadio Renato Dall'Ara                         | 36,000   |
| Cagliari      | Cagliari | Unipol Domus                                   | 16,416   |
| Como          | Como     | Stadio Giuseppe Sinigaglia                     | 13,602   |
| Empoli        | Empoli   | Stadio Carlo Castellani - Computer Gross Arena | 16,167   |
| Fiorentina    | Florence | Stadio Artemio Franchi                         | 43,118   |
| Genoa         | Genoa    | Stadio Luigi Ferraris                          | 33,205   |
| Hellas Verona | Verona   | Stadio Marcantonio Bentegodi                   | 31,713   |
| Inter Milan   | Milan    | San Siro                                       | 75,710   |
| Juventus      | Turin    | Juventus Stadium                               | 41,507   |
| Lazio         | Rome     | Stadio Olimpico                                | 67,585   |
| Lecce         | Lecce    | Stadio Via del mare                            | 30,354   |
| AC Milan      | Milan    | San Siro                                       | 75,710   |
| Monza         | Monza    | Stadio Brianteo                                | 17,102   |
| Napoli        | Naples   | Stadio Diego Amando Maradona                   | 54,732   |
| Parma         | Parma    | Stadio Ennio Tardini                           | 22,352   |
| Roma          | Rome     | Stadio Olimpico                                | 67,585   |
| Torino        | Turin    | Stadio Olimpico Grande Torino                  | 28,177   |
| Udinese       | Udine    | Stadio Friuli                                  | 25,132   |
| Venezia       | Venice   | Stadio Pier Luigi Penzo                        | 11,150   |

**unbeaten run** (21 matches)  
**Longest winless run** Empoli (20 matches)  
**Longest losing run** Monza (6 matches)  
**Highest attendance** 75,502  
**Lowest attendance** AC Milan 0–0 Juventus (23 November 2024)<sup>[2]</sup>  
**Total attendance** 11,719,935<sup>[2]</sup>  
**Average attendance** 30,842<sup>[2]</sup>

← 2023–24 2025–26 →



Locations of the 2024–25 Serie A teams

# BEAUTIFUL SOUP EXAMPLE

```
# find all H2 tags and find the one contains "stadium"
target_header = None
for h2 in soup.find_all("h2"):
    if "Stadiums" in h2.get_text():
        target_header = h2
        break

#find the related tables
table = target_header.find_next("table",
    class_="wikitable")
```

```
#extract data
rows = []
for tr in table.find_all("tr")[1:]: #skipe first row
    cells = [td.get_text(strip=True) for td in tr.find_all("td")]
    if len(cells) >= 4:
        rows.append({
            "Team": cells[0],
            "Location": cells[1],
            "Stadium": cells[2],
            "Capacity": cells[3]
        })

df = pd.DataFrame(rows)
print(df)
```

# BEAUTIFOUL SOUP EXAMPLE

- There is no h2 tag
- There are h3 only..

|    | Team          | Location | Stadium                                        |
|----|---------------|----------|------------------------------------------------|
| 0  | Atalanta      | Bergamo  | Gewiss Stadium                                 |
| 1  | Bologna       | Bologna  | Stadio Renato Dall'Ara                         |
| 2  | Cagliari      | Cagliari | Unipol Domus                                   |
| 3  | Como          | Como     | Stadio Giuseppe Sinigaglia                     |
| 4  | Empoli        | Empoli   | Stadio Carlo Castellani - Computer Gross Arena |
| 5  | Fiorentina    | Florence | Stadio Artemio Franchi                         |
| 6  | Genoa         | Genoa    | Stadio Luigi Ferraris                          |
| 7  | Hellas Verona | Verona   | Stadio Marcantonio Bentegodi                   |
| 8  | Inter Milan   | Milan    | San Siro                                       |
| 9  | Juventus      | Turin    | Juventus Stadium                               |
| 10 | Lazio         | Rome     | Stadio Olimpico                                |
| 11 | Lecce         | Lecce    | Stadio Via del mare                            |
| 12 | AC Milan      | Milan    | San Siro                                       |
| 13 | Monza         | Monza    | Stadio Brianteo                                |
| 14 | Napoli        | Naples   | Stadio Diego Armando Maradona                  |
| 15 | Parma         | Parma    | Stadio Ennio Tardini                           |
| 16 | Roma          | Rome     | Stadio Olimpico                                |
| 17 | Torino        | Turin    | Stadio Olimpico Grande Torino                  |
| 18 | Udinese       | Udine    | Stadio Friuli                                  |
| 19 | Venezia       | Venice   | Stadio Pier Luigi Penzo                        |



# TRY YOURSELF

- Download the first 20 national male teams according to the fifa ranking
- [https://en.wikipedia.org/wiki/FIFA\\_Men%27s\\_World\\_Ranking](https://en.wikipedia.org/wiki/FIFA_Men%27s_World_Ranking)

```
from pandas.io import html
```

```
from io import StringIO
```

```
tables = soup.find_all("table", class_="wikitable")
```

```
ranking_html = str(tables[1])
```

```
html_buffer = StringIO(ranking_html)
```

```
df = html.read_html(html_buffer, flavor="lxml")[0]
```

## FIFA Men's World Ranking

61 languages

Article Talk

Read View source View history Tools

From Wikipedia, the free encyclopedia

*This article is about the men's rankings. For the women's rankings, see FIFA Women's World Ranking.*

The **FIFA Men's World Ranking** is a ranking system for men's national teams in association football, first introduced in December 1992.<sup>[1]</sup> The men's teams of the member nations of FIFA, football's world governing body, are ranked based on their game results with the most successful teams being ranked the highest. As of October 2025 the rankings were led by Spain. Eight teams (Argentina, Belgium, Brazil, France, Germany, Italy, the Netherlands and Spain) have held the top position, of which Brazil have spent the longest time ranked first.

A points system is used, with points being awarded based on the results of all FIFA-recognised full international matches. The ranking system has been revamped on several occasions, generally responding to criticism that the preceding calculation method did not effectively reflect the relative strengths of the national teams. Since 16 August 2018, the ranking system has adopted the [Elo rating system](#) used in chess and Go.

The ranking is sponsored by [Coca-Cola](#); as such, the **FIFA/Coca-Cola World Ranking** name is also used. Coca-Cola also sponsors the [women's counterpart](#).

### Current calculation method

In September 2017, FIFA announced they were reviewing the ranking system and would decide after the end of the [2018 FIFA World Cup qualification](#) whether any changes would be made to improve the ranking.<sup>[2]</sup> FIFA announced on 10 June 2018 that the ranking system would be updated following the [2018 World Cup finals](#). The calculation method adopted closely modelled the [Elo rating system](#) (see [calculation method](#)). The weighting designated for each confederation for ranking purposes was abolished.<sup>[3]</sup> The new methodology does not account for home or away games and margin of the victory, as in several popular unofficial Elo-based



Top 20 rankings as of 17 October 2025<sup>[1]</sup>

| Rank | Change | Team                                   | Points  |
|------|--------|----------------------------------------|---------|
| 1    | ▲ 1    | <span><span></span></span> Spain       | 1875.37 |
| 2    | ▲ 1    | <span><span></span></span> France      | 1870.92 |
| 3    | ▼ 2    | <span><span></span></span> Argentina   | 1870.32 |
| 4    | —      | <span><span></span></span> England     | 1820.44 |
| 5    | ▲ 1    | <span><span></span></span> Portugal    | 1779.55 |
| 6    | ▼ 1    | <span><span></span></span> Brazil      | 1761.6  |
| 7    | —      | <span><span></span></span> Netherlands | 1754.17 |
| 8    | —      | <span><span></span></span> Belgium     | 1739.54 |
| 9    | ▲ 1    | <span><span></span></span> Croatia     | 1714.2  |
| 10   | ▲ 1    | <span><span></span></span> Italy       | 1710.06 |
| 11   | ▲ 1    | <span><span></span></span> Morocco     | 1706.27 |

# EXERCISE

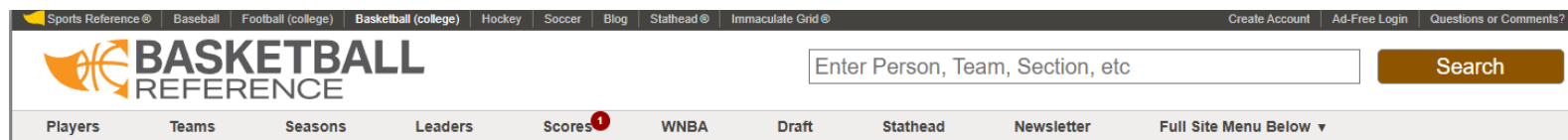
- Consider the web site <http://quotes.toscrape.com>
- Let see the exercise1\_confusion\_solution.ipynb
- Taking inspiration from the exercise1\_confusion\_solution.ipynb file you must:
- Scrape 5 pages of quotes or more...(using a loop...)
- For each quote, scrape:
  - author
  - text
  - tags
- Create a DataFrame which contains the quotes and their details

# EXERCISE

Scrape the author's details navigating to the authors' pages:

- For each author, scrape:
  - name
  - birthday
  - location
  - description
- Add these new data to the DataFrame
- Try to scrape books from <http://books.toscrape.com/> getting all the book displayed details

# TRY YOURSELF (NOT WITH JUPYTER)



- Extract the latest gamelog of a specific player (e.g. Stephen Curry) from this site
- <https://www.basketball-reference.com/>

A screenshot of the player profile page for Stephen Curry on Basketball Reference. The page includes Curry's photo, name, pronunciation, and various career highlights. A "More bio, uniform, draft info" button is visible. At the bottom, there is a "SUMMARY" table showing his 2024-25 career statistics.

**Stephen Curry**  
Pronunciation: 'ISTEFF-in'  
**Wardell Stephen Curry II** • Instagram: [stephencurry30](#)  
(Steph, Baby-Faced Assassin, Chef Curry, The Human Torch, Threezus)  
Position: Point Guard • Shoots: Right  
6-2, 185lb (188cm, 83kg)  
Team: [Golden State Warriors](#)

[More bio, uniform, draft info ▼](#)

| SUMMARY | G    | PTS  | TRB | AST | FG%  | FG3% | FT%  | eFG% | PER  | WS    |
|---------|------|------|-----|-----|------|------|------|------|------|-------|
| 2024-25 | 70   | 24.5 | 4.4 | 6.0 | 44.8 | 39.7 | 93.3 | 57.2 | 21.5 | 7.9   |
| Career  | 1026 | 24.7 | 4.7 | 6.4 | 47.1 | 42.3 | 91.1 | 58.2 | 23.5 | 143.1 |

Preview:

|   | metric | label          | value_primary | value_secondary | section |
|---|--------|----------------|---------------|-----------------|---------|
| 0 | G      | Games          | 2024-25       | Career          |         |
| 1 | G      | Games          | 70            | 1026            | p1      |
| 2 | PTS    | Points         | 24.5          | 24.7            | p1      |
| 3 | TRB    | Total Rebounds | 4.4           | 4.7             | p1      |
| 4 | AST    | Assists        | 6.0           | 6.4             | p1      |



# DYNAMIC WEB SCRAPING



# HOW WE CAN EXTRACT DATA DYNAMICALLY FROM WEB PAGES?

- Performing web scraping with:
  - Text pattern matching
  - HTML parsing
  - DOM parsing
  - ...
- We will use the Selenium Python library to interact with Selenium WebDriver, enabling dynamic web scraping

# SELENIUM WEBDRIVER

- WebDriver drives a browser natively, as a user would, either locally or on a remote machine using the Selenium server, marks a leap forward in terms of browser automation.
- Selenium WebDriver refers to both the language bindings and the implementations of the individual browser controlling code. This is commonly referred to as just WebDriver.
- Selenium WebDriver is a W3C Recommendation
- WebDriver is designed as a simple and more concise programming interface
- WebDriver is a compact object-oriented API
- It drives the browser effectively.
- Source: <https://www.selenium.dev/documentation/webdriver/>
- Definition: A powerful tool for web automation and dynamic web scraping.
- Purpose: Facilitates interaction with web browsers through programmatic control, ideal for testing and scraping dynamic web content.
- Key features:
  - Browser Automation: Control browser actions like navigating pages, clicking links, and submitting forms
  - Support for Multiple Browsers: Compatible with popular browsers like Chrome, Firefox, Safari, etc.
  - Interaction with Dynamic Elements: Handles JavaScript and AJAX-loaded content effectively.
- further details at: <https://selenium-python.readthedocs.io/index.html>

# SELENIUM EXAMPLE

```
from selenium import webdriver

# Launch the browser
driver = webdriver.Chrome() # Replace with the browser of your choice

# Navigate to a webpage
driver.get('https://example.com')

# Perform actions and extract data
title = driver.title
print(title)

# Close the browser
driver.quit()
```

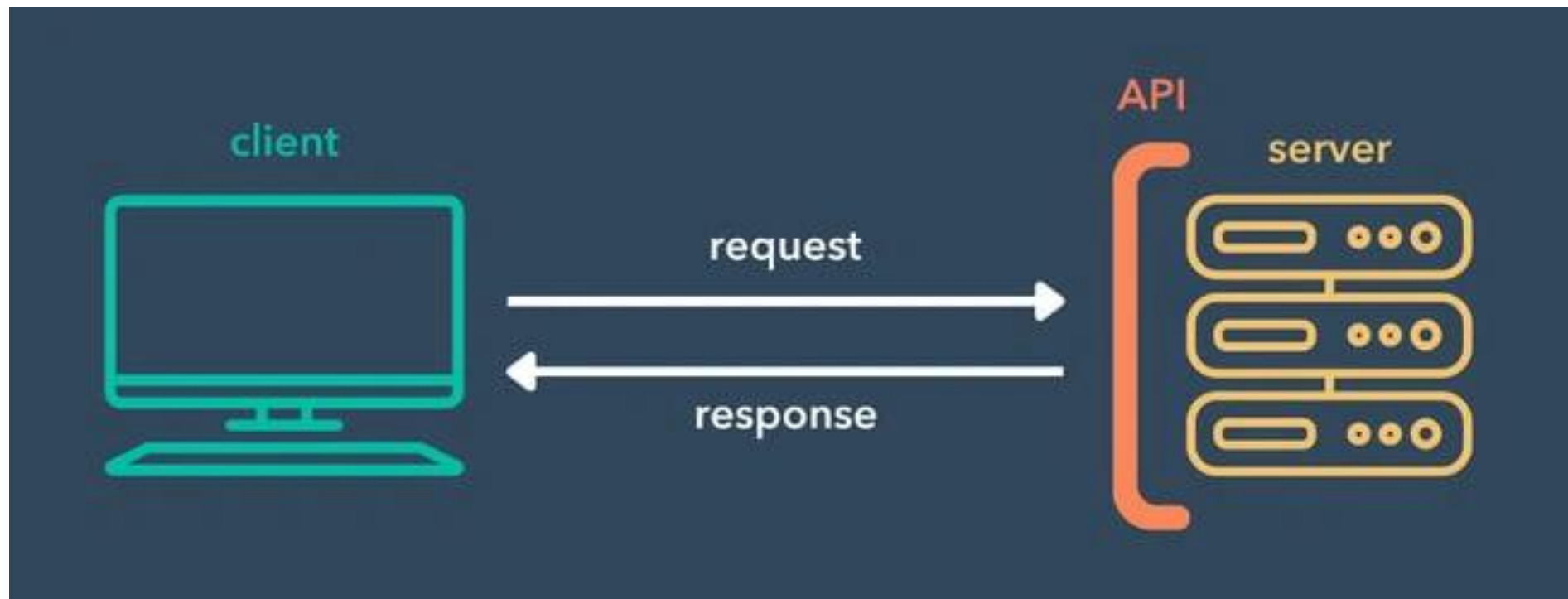
- Let see test\_selenium.ipynb



# EXERCISE

- Perform the same exercises, but use Selenium instead of requests.get
- Let consider <https://quotes.toscrape.com/js/>
- Extracts, for each quote:
  - the text of the quote
  - the author
  - the tags
- Navigates through the site using the “Next” button
- Stops after 3 pages (or when there is no “Next” button)

# API



## SO, A WEB API IS...

- A Web API is an API over the web which can be accessed using HTTP protocol
- Many large websites and services such as Google Maps, Twitter, Facebook, PayPal, etc. provide APIs allowing developers to make use of their data (e.g. displaying your twitter stream on your blog) or services (e.g. using Facebook login to log in your users)
- Very often you will deal with HTTP REST APIs... that are Web APIs that follow (or at least they try to...) a design paradigm and its principles called REST (REpresentational State Transfer)

# HOW CAN I EXTRACT DATA FROM A WEB API?

- Using third-party libraries like Spotipy (API from Spotify) or using web clients to perform HTTP requests and consume their responses
- We will use a Python library called requests:
  - widespread usage and community
  - well documented, many examples
  - used by plenty of other libraries and developers

# API CALL

- <https://catfact.ninja/>

```
def get_cat_facts(num_facts=3):  
    url = "https://catfact.ninja/facts"  
    params = {  
        "limit": num_facts  
    }  
    try:  
        response = requests.get(url, params=params, timeout=5)  
        response.raise_for_status()  
        data = response.json()  
        print(f"Here {num_facts} facts over cats:\n")  
        for i, fact in enumerate(data.get("data", []), start=1):  
            print(f"{i}. {fact['fact']}\n")
```

- See api.ipynb

The screenshot shows a REST client interface for the `GET /facts` endpoint. The description is "Returns a list of facts". There is a "Try it out" button in the top right corner.

**Parameters**

| Name                                                                   | Description                                                                |
|------------------------------------------------------------------------|----------------------------------------------------------------------------|
| <code>max_length</code><br><small>integer(\$int64)<br/>(query)</small> | maximum length of returned fact<br><input type="text" value="max_length"/> |
| <code>limit</code><br><small>integer(\$int64)<br/>(query)</small>      | limit the amount of results returned<br><input type="text" value="limit"/> |

**Responses**

| Code | Description                                                                                                                                             | Links                    |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|
| 200  | successful operation<br><small>Media type: <input type="text" value="application/json"/><br/>Controls: Accept header<br/>Example Value   Schema</small> | <a href="#">No links</a> |

The response body for the 200 status code is shown in a dark box with the following JSON structure:

```
{  
  "fact": "string",  
  "length": 0  
}
```

## SOME APIS ENDPOINTS AVAILABLE

- We are going to explore APIs endpoint available online, which are:
  - OpenWeatherMap API
  - The Cat API
  - REST Countries API
  - JokeAPI
  - PokeAPI
  - News API
  - GitHub API
  - NASA API
  - Spotify API
  - OMDB API (Open Movie Database)
  - CoinGecko API
  - Unsplash API
  - TMDb API (The Movie Database)
- ....(please search for more API )

# API

## OpenWeatherMap API

- API details:
  - Description: Provides weather data including current weather, forecasts, and historical data
  - Key Features: Global coverage, different weather parameters, forecast data
  - Access Difficulty: Easy
  - Free Access: Yes (Limited to 60 calls/minute)
  - API Key Required: Yes
  - API Documentation URL: <https://openweathermap.org/api>

## The Cat API

- API details:
  - Description: Offers a variety of cat images and related data
  - Key Features: Random cat images, searchable by categories and breeds
  - Access Difficulty: Easy
  - Free Access: Yes
  - API Key Required: Yes
  - API Documentation URL: <https://thecatapi.com/>

# API

## REST Countries API

- API details:
  - Description: Provides information about countries including name, population, area, and more
  - Key Features: Filter by name, full name, code; access country flags and regional data
  - Access Difficulty: Very Easy
  - Free Access: Yes
  - API Key Required: No
  - API Documentation URL: <https://restcountries.com/>

## JokeAPI

- API details:
  - Description: A service for fetching programming and general jokes in various formats
  - Key Features: Supports various categories, including programming, miscellaneous, and dark jokes
  - Access Difficulty: Very Easy
  - Free Access: Yes
  - API Key Required: No
  - API Documentation URL: <https://jokeapi.dev/>



# API

## PokeAPI

- API details:
  - Description: Offers extensive data about Pokémon
  - Key Features: Data on Pokémon species, abilities, moves, items
  - Access Difficulty: Very Easy
  - Free Access: Yes
  - API Key Required: No
  - API Documentation URL: <https://pokeapi.co/>

## News API

- API details:
  - Description: Fetches the latest news and articles from various sources and blogs
  - Key Features: Real-time news, searchable by keywords, sources, domains
  - Access Difficulty: Easy
  - Free Access: Yes (Limited to 100 requests/day)
  - API Key Required: Yes
  - API Documentation URL: <https://newsapi.org/>

# EXERCISE

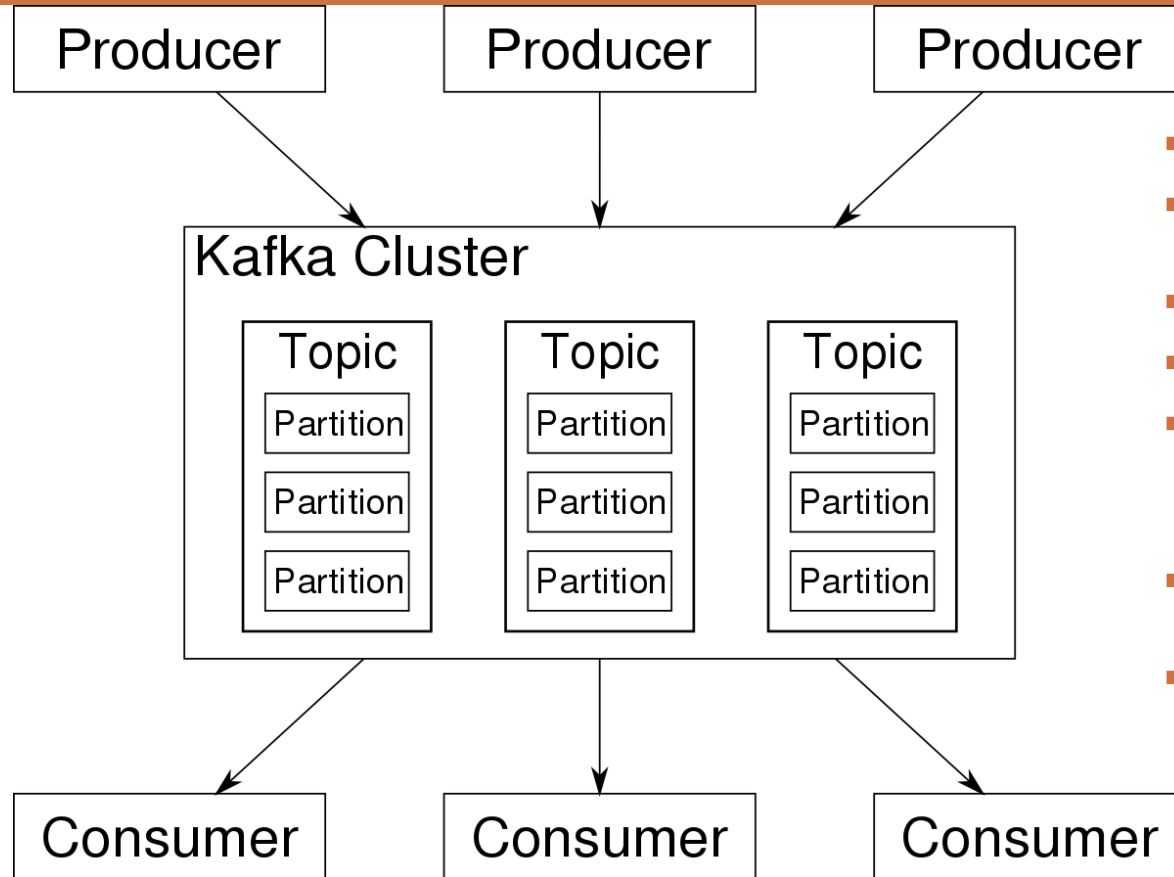
- Try to:
  - Use different APIs (with token, limitation)
  - Pass a dynamic parameter and handle “no results”.
    - <https://api.adviceslip.com/advice/search/{query}>
  - Fetch weather data, compute a statistic, save to CSV.
    - Geocoding (city → lat/lon): <https://geocoding-api.open-meteo.com/v1/search?name={city}&count=1>
    - Weather (hourly temps): [https://api.open-meteo.com/v1/forecast?latitude=..&longitude=..&hourly=temperature\\_2m](https://api.open-meteo.com/v1/forecast?latitude=..&longitude=..&hourly=temperature_2m)



KAFKA



# KAFKA



- Producers: are application/components that write messages to Kafka
- Consumers: are application/components that receive messages from Kafka
- Topics:
- Messages are sent to given topics in Kafka
- Consumers subscribe topics and they receive messages only from the topics they are subscribed to
- If you run in your local notebook, you should install kafka in somehow, like local installation or through Docker.
- Moreover, for local installation, remember to install package kafka-python (client) with command: `python -m pip install kafka-python`

# INTEL LAB DATA

- Intel Lab Data Collection: Data from 54 sensors in the Intel Berkeley Research lab
- Duration: Collected from February 28th to April 5th, 2004
- Data Points: Approximately 2.3 million sensor readings
- Measurements: Includes temperature, humidity, light, and
- \*<https://db.csail.mit.edu/labdata/labdata.html>
  - Simulation of Intel Lab Data is done using a Python script! voltage
- The Intel Lab Data dataset comprises various sensor readings, each characterized by the following fields (with their respective units of measure):
  - Date: Format (yyyy-mm-dd)
  - Time: Format (hh:mm:ss.xxx)
  - Epoch: Sequence number (integer)
  - MotelID: Sensor ID (1-54 range)
  - Temperature: Degrees Celsius (°C)
  - Humidity: Percentage (%)
  - Light: Lux (intensity)
  - Voltage: Volts (V, 2-3V range)

# THE PRODUCER

- Pip install kafka
- The producer

```
from kafka import KafkaProducer
```

```
TOPIC = "sensors"
```

```
producer = KafkaProducer(  
    bootstrap_servers="localhost:9092",  
    value_serializer=lambda v: json.dumps(v).encode("utf-8")  
)
```

```
try:
```

```
    print("Sending:", msg)
```

```
    producer.send(TOPIC, msg)
```

```
    producer.flush()
```

```
except Exception as e:
```

```
    print("Error fetching/sending data:", e)
```

# THE CONSUMER

```
from kafka import KafkaConsumer
```

```
TOPIC = "sensors"
```

```
consumer = KafkaConsumer(
```

```
    TOPIC,
```

```
    bootstrap_servers="localhost:9092",
```

```
    value_deserializer=lambda v: json.loads(v.decode("utf-8")),
```

```
    auto_offset_reset="latest",
```

```
    enable_auto_commit=True,
```

```
    group_id="lab-group"
```

```
)
```

```
print("Waiting for messages...")
```

```
for msg in consumer:
```

```
    data = msg.value
```

```
    ....
```

- Let see the example

# CONFLUENT

```
!pip install confluent-kafka
from confluent_kafka import Producer

TOPIC=«sensors»

producer=Producer({'bootstrap.servers':'localhost:9092'})

producer.produce(TOPIC,value=json.dumps(msg).encode('utf-8'))

producer.flush()
```

```
consumer=Consumer({'bootstrap.servers':'localhost:9092',
'auto.offset.reset':'latest',
'group.id':'lab-group'})
consumer.subscribe([TOPIC])
While True
    msg=consumer.poll(1.0)
    If msg is None:
        continue
    data=json.loads(msg.value().decode('utf-8'))
    print(data)
```



## EXERCISE

- Collect crypto data from CoinGecko
- <https://www.coingecko.com/en/api>

```
API_URL =  
"https://api.coingecko.com/api/v3/simple/price"  
  
PARAMS = {  
    "ids": "bitcoin,ethereum",  
    "vs_currencies": "eur"  
}  
  
resp = requests.get(API_URL, params=PARAMS,  
                    timeout=10)
```